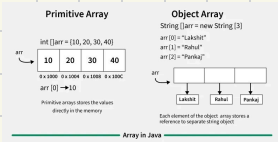


Arrays

An **array** is a **linear data structure** that stores a **fixed number of elements** of the **same data type** in **contiguous memory locations**.

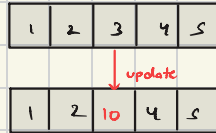


2) Up daton

Syntax :- `arrayName [index] = value;`

Example

`arr [2] = 10;`



Example

① `int[] arr = {1, 2, 3, 4, 5};`

② `char[] arr = {'a', 'e', 'i', 'o', 'u'};`



3) Traversal / print Array

Array Traversal

Visit and process each element of the array, one by one, from start to end.

Index → 0 1 2 3 4
 Array → 10 20 30 40 50
`int[] arr = {10, 20, 30, 40, 50};`

1. Using for Loop (Recommended)

```
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

Traversal Steps

i = 0: 10 20 30 40 50
 i = 1: 10 20 30 40 50
 i = 2: 10 20 30 40 50
 i = 3: 10 20 30 40 50
 ...

Each element is visited exactly once.

2. Using Enhanced for Loop (Simpler, read-only access)

```
for (int value : arr) {
    System.out.println(value);
}
```

Traversal Steps

value = 10: 10 20 30 40 50
 value = 20: 10 20 30 40 50
 value = 30: 10 20 30 40 50
 value = 40: 10 20 30 40 50
 ...

Automatically gives each element.

3. Using while Loop (Manual control over index)

```
int i = 0;
while (i < arr.length) {
    System.out.println(arr[i]);
    i++;
}
```

Traversal Steps

i = 0: 10 20 30 40 50
 i = 1: 10 20 30 40 50
 i = 2: 10 20 30 40 50
 i = 3: 10 20 30 40 50
 ...

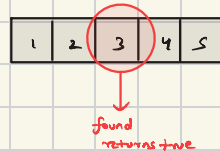
Index is incremented manually.

Result: All elements are visited in order: 10 20 30 40 50

4) Searching

```
private static boolean CheckIfPresent(int[] a, int value){
    for (int i = 0; i < a.length; i++){
        if(a[i] == value){
            return true;
        }
    }
    return false;
}
```

Example



Search for 3 in the array

Standard Questions

Q1. Two Sum

You are given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`. You may assume that each input would have exactly one solution, and you may not use the same element twice. You can return the answer in any order.

Code:-

Example 1:

Input: nums = [2,7,11,15], target = 9

Output: [0,1]

Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].

Example 2:

Input: nums = [3,2,4], target = 6

Output: [1,2]

Example 3:

Input: nums = [3,3], target = 6

Output: [0,1]

```
public int[] twoSum(int[] nums, int target) {
    int[] result = new int[2];

    for(int i = 0; i < nums.length; i++){
        for(int j = i + 1; j < nums.length; j++){
            if(nums[i] + nums[j] == target){
                result[0] = i;
                result[1] = j;
            }
        }
    }

    return result;
}
```

Dry Run

Dry Run: Two Sum (Brute Force)

```

Code
public static int[] twoSum(int[] nums, int target) {
    int[] result = new int[2];
    for (int i = 0; i < nums.length; i++) {
        for (int j = i + 1; j < nums.length; j++) {
            if (nums[i] + nums[j] == target) {
                result[0] = i;
                result[1] = j;
            }
        }
    }
    return result;
}

```

Input

nums = [2, 7, 11, 15]
target = 9

Index	0	1	2	3
Value	2	7	11	15

Initialization

int[] result = new int[2];
result = [0, 0] (default values)

Goal

Find two indices i and j such that:
nums[i] + nums[j] == target
Return the indices as an array.

Expected Output

For nums = [2, 7, 11, 15] and target = 9:
Output: [0, 1]
(Because 2 + 7 = 9)

Step-by-Step Dry Run

i = 0, j = 1	i = 0, j = 2	i = 0, j = 3	i = 1, j = 2	i = 1, j = 3	i = 2, j = 3
0 1 2 3 2 7 11 15 ↑ ↑ 13 → 9 ✓ result[0] = 0 result[1] = 1	0 1 2 3 2 7 11 15 ↑ ↑ 17 → 9 ✗ No change	0 1 2 3 2 7 11 15 ↑ ↑ 17 → 9 ✗ No change	0 1 2 3 2 7 11 15 ↑ ↑ 22 → 9 ✗ No change	0 1 2 3 2 7 11 15 ↑ ↑ 26 → 9 ✗ No change	0 1 2 3 2 7 11 15 ↑ ↑ 26 → 9 ✗ No change

Final Result

The function returns: **[0, 1]**
These are the indices of the two numbers (2 and 7) whose sum is equal to 9.

Conclusion

- The pair is found in the first iteration itself (i = 0, j = 1).
- The loops continue, but no other pair matches the target.
- Final returned array: **[0, 1]**.

Q2. Remove Element

Given an integer array nums and an integer val, remove all occurrences of val in nums in-place. The order of the elements may be changed. Then return the number of elements in nums which are not equal to val. Consider the number of elements in nums which are not equal to val be k, to get accepted, you need to do the following things:

- Change the array nums such that the first k elements of nums contain the elements which are not equal to val. The remaining elements of nums are not important as well as the size of nums.
- Return k.

Example 1:

Input: nums = [3,2,2,3], val = 3

Output: 2, nums = [2,2,_,_]

Explanation: Your function should return k = 2, with the first two elements of nums being 2.

It does not matter what you leave beyond the returned k (hence they are underscores).

Example 2:

Input: nums = [0,1,2,2,3,0,4,2], val = 2

Output: 5, nums = [0,1,4,0,3,_,_]

Explanation: Your function should return k = 5, with the first five elements of nums containing 0, 0, 1, 3, and 4.

Note that the five elements can be returned in any order.

It does not matter what you leave beyond the returned k (hence they are underscores).

Code:-

```
public int removeElement(int[] nums, int val) {
    int slow = 0;

    for(int fast = 0; fast < nums.length; fast++){
        if(nums[fast] != val) nums[slow++] = nums[fast];
    }

    return slow;
}
```

Dry Run

Dry Run: Remove Element (Two Pointers)

```

Code
public int removeElement(int[] nums, int val) {
    int slow = 0;
    for (int fast = 0; fast < nums.length; fast++) {
        if (nums[fast] != val) {
            nums[slow] = nums[fast];
            slow++;
        }
    }
    return slow;
}

```

Input

nums = [3, 2, 2, 3], val = 3

Goal

Remove all occurrences of val from nums in-place. Return the number of elements that are not equal to val.

Step-by-Step Dry Run

Step	fast	slow	nums[fast]	nums[slow]	slow++
1	0	0	3	3	1
2	1	0	2	2	1
3	2	1	2	2	2
4	3	2	3	3	3

Final Result

slow = 2
nums = [2, 2, _, _]

Output (from main)

slow = 2
nums = [2, 2, _, _]

Key Takeaways

- Two pointers approach.
- slow pointer moves only when a valid element is found.
- fast pointer iterates through all elements.
- Non-removable O(1) space.

Q3. Best Time to Buy and Sell Stock

You are given an array prices where prices[i] is the price of a given stock on the ith day. You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock. Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Example 1:
Input: nums = [3,2,2,3], val = 3
Output: 2, nums = [2,_,_,_]
Explanation: Your function should return k = 2, with the first two elements of nums being 2.
It does not matter what you leave beyond the returned k (hence they are underscores).

Example 2:
Input: nums = [0,1,2,2,3,0,4,2], val = 2
Output: 5, nums = [0,1,4,0,3,_,_,_]
Explanation: Your function should return k = 5, with the first five elements of nums containing 0, 0, 1, 3, and 4.
Note that the five elements can be returned in any order.
It does not matter what you leave beyond the returned k (hence they are underscores).

Code :-

Dry Run

```
public static int maxProfit(int[] prices) {
    int minPrice = prices[0];
    int maxProfit = 0;

    for (int i = 1; i < prices.length; i++) {

        if (prices[i] < minPrice) {
            minPrice = prices[i];
        }

        if (prices[i] - minPrice > maxProfit) {
            maxProfit = prices[i] - minPrice;
        }
    }

    return maxProfit;
}
```

Dry Run: Best Time to Buy and Sell Stock (Single Transaction)

Code

```
public static int maxProfit(int[] prices) {
    int minPrice = prices[0];
    int maxProfit = 0;
    for (int i = 1; i < prices.length; i++) {
        if (prices[i] < minPrice) {
            minPrice = prices[i];
        }
        if (prices[i] - minPrice > maxProfit) {
            maxProfit = prices[i] - minPrice;
        }
    }
    return maxProfit;
}
```

Input

prices = [7, 1, 5, 3, 6, 4]

Index	0	1	2	3	4	5
Price	7	1	5	3	6	4

Goal

Find the maximum profit by buying at a lower price and selling at a higher price (buy before sell).

Key Idea

- Keep track of the minimum price seen so far (buy price).
- For each price, calculate the profit if we sell today (prices[i] - minPrice).
- Update the maximum profit if today's profit is higher.

Initial State

minPrice = prices[0] = 7
maxProfit = 0

We start from index 1 because minPrice is already set to prices[0].

Step-by-Step Dry Run

Iteration	Current Price (prices[i])	minPrice (buyPrice)	maxProfit (sellPrice)	Check 1 (prices[i] < minPrice)	minPrice (after)	Check 2 (prices[i] - minPrice > maxProfit)	maxProfit (after)	Explanation
0	-	-	0	-	-	-	-	Initial state
1	1	7	0	1 < 7 → Yes	1	1 - 1 < 0 → No	0	New minimum price found (1)
2	5	1	0	5 < 1 → No	1	5 - 1 > 0 → Yes	4	New max profit = 4 (Buy at 1, Sell at 5)
3	3	1	4	3 < 1 → No	1	3 - 1 < 4 → No	4	Profit = 2 (not greater than 4)
4	6	1	4	6 < 1 → No	1	6 - 1 > 4 → Yes	5	New max profit = 5 (Buy at 1, Sell at 6)
5	4	1	5	4 < 1 → No	1	4 - 1 < 5 → No	5	Profit = 3 (not greater than 5)

Result

Max Profit :- 5

Buy at price 1 (index 1) and sell at price 6 (index 4).

Visual Representation

Index	0	1	2	3	4	5
Price	7	1	5	3	6	4

↑ Buy (price = 1)
↑ Sell (price = 6)

Complexity

- Time Complexity: O(n)
We traverse the array once.
- Space Complexity: O(1)
We only use a few variables.

Q4. Best Time to Buy and Sell Stock - II

You are given an integer array prices where prices[i] is the price of a given stock on the ith day. On each day, you may decide to buy and/or sell the stock. You can only hold at most one share of the stock at any time. However, you can sell and buy the stock multiple times on the same day, ensuring you never hold more than one share of the stock. Find and return the maximum profit you can achieve.

Example 1:
Input: nums = [3,2,2,3], val = 3
Output: 2, nums = [2,_,_,_]
Explanation: Your function should return k = 2, with the first two elements of nums being 2.
It does not matter what you leave beyond the returned k (hence they are underscores).

Example 2:
Input: nums = [0,1,2,2,3,0,4,2], val = 2
Output: 5, nums = [0,1,4,0,3,_,_,_]
Explanation: Your function should return k = 5, with the first five elements of nums containing 0, 0, 1, 3, and 4.
Note that the five elements can be returned in any order.
It does not matter what you leave beyond the returned k (hence they are underscores).

Code :-

Dry Run

```
public int maxProfit(int[] prices) {
    int totalProfit = 0;

    for(int i = 0; i < prices.length-1; i++){
        if(prices[i] < prices[i+1]) totalProfit += prices[i+1] - prices[i];
    }

    return totalProfit;
}
```

Dry Run: Best Time to Buy and Sell Stock II (Multiple Transactions)
(Your Code Run on all positive differences)

Code

```
public static int maxProfit(int[] prices) {
    int totalProfit = 0;
    for (int i = 0; i < prices.length - 1; i++) {
        if (prices[i] < prices[i + 1]) {
            totalProfit += prices[i + 1] - prices[i];
        }
    }
    return totalProfit;
}
```

Input

prices = [2, 1, 5, 3, 6, 4]

Index	0	1	2	3	4	5
Price	2	1	5	3	6	4

Goal

Find the maximum possible profit by making as many transactions as you like (Buy before sell).
Add all positive differences between consecutive days.

Step-by-Step Dry Run

Iteration	prices[i]	prices[i + 1]	Is prices[i] < prices[i + 1]	Profit added (if any)	totalProfit (after)	Explanation
0	2	1	2 < 1 → No	0	0	Price decreased, no profit.
1	1	5	1 < 5 → Yes	5 - 1 = 4	4	Buy at 1, Sell at 5 → profit = 4
2	5	3	5 < 3 → No	0	4	Price decreased, no profit.
3	3	6	3 < 6 → Yes	6 - 3 = 3	7	Buy at 3, Sell at 6 → profit = 3
4	6	4	6 < 4 → No	0	7	Price decreased, no profit.

Final Result

Total Profit :- 7

Transactions made:

- Buy at 1, Sell at 5 → Profit = 4
- Buy at 3, Sell at 6 → Profit = 3

Total Profit = 4 + 3 = 7

Visual Representation

prices = [2, 1, 5, 3, 6, 4]

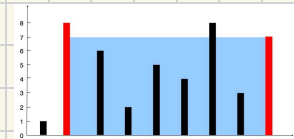
Key Takeaways

- We add the profit for every increasing difference between consecutive days.
- This allows multiple transactions (Buy and hold multiple times).
- Greedy approach: Buy every time the price is lower than the next day's price.
- Time Complexity: O(n)
- Space Complexity: O(1)

Q5. Container with Most Water

You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the ith line are (i, 0) and (i, height[i]). Find two lines that together with the x-axis form a container, such that the container contains the most water. Return the maximum amount of water a container can store. Notice that you may not slant the container.

Example 1:



Example 2:
Input: height = [1,1]
Output: 1

Dry Run

Code:-

```
public int maxArea(int[] height) {
    int start = 0;
    int end = height.length - 1;
    int Area = 0;

    while (start < end) {
        int currentWidth = end - start;
        int currentHeight = Math.min(height[start], height[end]);
        int currentArea = currentWidth * currentHeight;

        Area = Math.max(Area, currentArea);

        if (height[start] < height[end]) {
            start++;
        } else {
            end--;
        }
    }

    return Area;
}
```

Q6. Rotate Array

Given an integer array nums, rotate the array to the right by k steps, where k is non-negative.

Example 1:
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]

Example 2:
Input: nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]

Code:-

```
public void swap(int[] arr,int start,int end){
    while(start < end){
        int temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
        start++;
        end--;
    }
}

public void rotate(int[] nums, int k) {
    int n = nums.length;
    k = k%n;

    swap(nums,0,n-1);
    swap(nums,0,k-1);
    swap(nums,k,n-1);
}
```

Dry Run

Dry Run: Rotate Array to the Right by k (k = 3)

Input: nums = [1, 2, 3, 4, 5, 6, 7], k = 3

Step 1: Initial array [1, 2, 3, 4, 5, 6, 7]. k = 3.

Step 2: Rotate the array by 1 step to the right. Result: [7, 1, 2, 3, 4, 5, 6].

Step 3: Rotate the array by 2 steps to the right. Result: [6, 7, 1, 2, 3, 4, 5].

Step 4: Rotate the array by 3 steps to the right. Result: [5, 6, 7, 1, 2, 3, 4].

Final Result: [5, 6, 7, 1, 2, 3, 4]